P1 - The 2024 Final Lab Exam (Python Part), explained line by line

Sales data -> pandas -> matplotlib -> SQLite -> PyTorch. 40 marks. This is the paper your mock will most closely resemble. Full runnable file: `Solutions_py/2024_sales_sqlite_pytorch_solution.py`*. Extra "what if the examiner changes it" drills:* `Solutions_py/2024_twist_drills.py`*.*

> **Version 2 note (read first).** Your instructor's checklist for the practical assessment asks for a **regression** network (continuous target, `y = X @ w + b + noise`, Linear + ReLU, **no sigmoid**, seed set first) plus pandas merge, box plots, `UPDATE`, `to_sql` and exporting query results to CSV. The instructor also says your test is "very similar to the 2024/25 paper", so be ready for **either** network (the sigmoid one below, or the regression one): do exactly what the paper specifies, the training loop is identical. The 2024 paper below has none of the checklist extras, so use P1 to **understand the pipeline and how marks are earned**, and use T9 / T4-T7 / the Cheat Sheet for the checklist operations. Do not try to memorise this solution: practise its operations one at a time.

1. The paper at a glance

Task	What you build	Marks	Skills it tests
1a	Load CSV, fill blanks, make `Date` a real date, fix `Total`	2 + 3 + 2 + 2 = 9	`read_csv`, `fillna`, `to_datetime`, column maths
1b	Bar chart (qty per product) + line chart (sales per month, 2023)	2 + 2 = 4	`groupby`, `matplotlib`
2a	SQLite DB `SalesDB`, table `Sales`, insert with **no duplicates**	2 + 3 + 4 = 9	`sqlite3`, `CREATE TABLE`, `INSERT`
2b	Two queries: total 2023 sales; qty per product 2023, descending	3 + 3 = 6	`SUM`, `WHERE`, `GROUP BY`, `ORDER BY`
3a-3d	`SimpleNN` (10 -> 5 -> 3 -> 1), train 20 epochs, plot loss	4 + 2 + 3 + 3 = 12	`torch.nn`, training loop

Time plan: the 2024 paper was longer; your mid-term test is **ONE HOUR** (see `Test_Day_Card_60_Minutes`): read 3 min, Task 1 about 19, Task 2 about 18, Task 3 about 12, assemble + clean run 4, upload by minute 56.

Submission: ONE file called `your_name_python_task.py`. All three tasks in the same file, in order. The CSV is in the same folder as your script.

2. First 60 seconds: LOOK at the data before coding

Never clean data you have not looked at. Put this at the top of your script while exploring (delete or keep as prints):

```
df = pd.read_csv("sales_data.csv")
print(df.shape)            # (151, 5)  -> 151 rows, 5 columns
print(df.dtypes)           # Date is 'object' (text!) - that is why Task 1a asks to convert it
print(df.isna().sum())     # counts blank cells per column
print(df.duplicated().sum())   # counts exact repeated rows
```

What this file secretly contains (found by running exactly these lines):

Trap	Where	What it means for you
6 blank `Product`	e.g. 24/02/2023	fill with `"Unknown"`
1 blank `Price`	12/03/2023 Chocolate (Total is 50, Qty 5)	**work it out**: Price = Total / Quantity = 10. Filling 0 would erase a real £50 sale
3 blank `Total`	e.g. 22/02/2023 Sweets	Total = Quantity x Price

Trap	Where	What it means for you
6 exact duplicate rows	04/04/2022 - 10/04/2022	needed for the "no duplicates" mark in 2a
Dates are `dd/mm/yyyy`	whole column	tell pandas the **day comes first**
Three years of data	2022 (29 rows), 2023 (100), 2024 (22)	Task 1b line chart and 2b queries are about **2023 only**

> **The big picture analogy.** You run a corner shop. **pandas** is your spreadsheet clerk who tidies the sales ledger. **matplotlib** is the person who draws the wall charts. **SQLite** is the filing cabinet where the tidy ledger is stored permanently. **PyTorch** is an intern who is learning to guess numbers by trial and error.

3. Task 1a - Load, clean, convert (9 marks)

```
df = pd.read_csv("sales_data.csv")
print("Rows, columns:", df.shape)
print(df.head())
```

- `import pandas as pd` (at the top of the file): fetches the pandas toolbox; `pd` is just a short nickname.
- `pd.read_csv("sales_data.csv")`: **opens the file in a spreadsheet that lives in memory.** The result, `df`, is a *DataFrame* = a table with named columns. Analogy: double-clicking the file in Excel.
- `df.shape`: (rows, columns). Quick sanity check that you loaded the right file.
- `df.head()`: the first 5 rows. Like peeking at the top of the spreadsheet.

```
# Text column: a missing product name becomes 'Unknown'
df["Product"] = df["Product"].fillna("Unknown")

# Quantity: a missing quantity can only sensibly be 0
df["Quantity"] = df["Quantity"].fillna(0)

# Price: if Total is known we can WORK OUT the price (Total / Quantity),
# otherwise fall back to 0.  (Never just blindly fill 0 first – that
# would wipe out a real sale when Total is later recalculated.)
df["Price"] = df["Price"].fillna(df["Total"] / df["Quantity"])
df["Price"] = df["Price"].fillna(0)

# Total: a missing total = Quantity * Price
df["Total"] = df["Total"].fillna(df["Quantity"] * df["Price"])

print("\nMissing values per column AFTER cleaning:")
print(df.isna().sum())
```

- `df["Product"]`: pull out ONE column (a *Series*). Square brackets + quoted column name.
- `.fillna("Unknown")`: "wherever a cell is blank (NaN), write this instead". It **returns a new column**, so we assign it back with `df["Product"] = ...`. **TRAP:** `df["Product"].fillna("Unknown")` on its own changes nothing, because you never stored the answer.
- `df["Total"] / df["Quantity"]`: pandas divides **row by row** (no loop!). So `fillna(df["Total"] / df["Quantity"])` means "blank price? use that row's Total divided by that row's Quantity". Analogy: the receipt says 5 chocolates cost £50, so one costs £10.
- The second `fillna(0)` is the safety net: if even Total was blank, price becomes 0.
- `df["Quantity"] * df["Price"]` is again row-by-row.
- `df.isna().sum()`: `isna()` gives True/False for every cell; `sum()` counts the Trues per column. All zeros afterwards = clean.

```
df["Date"] = pd.to_datetime(df["Date"], dayfirst=True)

# --- Make sure Total = Quantity x Price (2 marks) --------------------
wrong = df["Total"] != df["Quantity"] * df["Price"]
print("\nRows where Total was wrong:", wrong.sum())
df["Total"] = df["Quantity"] * df["Price"]

# --- Remove exact duplicate rows (needed for 'no duplicates' in 2a) ---
before = len(df)
df = df.drop_duplicates()
print("Duplicate rows removed:", before - len(df))
```

- `pd.to_datetime(..., dayfirst=True)`: converts text such as `"05/01/2023"` into a real calendar date. Without `dayfirst=True` pandas may read it as **May 1st** (US style). Alternative that is even more explicit: `pd.to_datetime(df["Date"], format="%d/%m/%Y")`. Analogy: turning the handwritten "5/1/23" on a receipt into a proper date stamp the computer can add and compare.
- `wrong = df["Total"] != ...; wrong.sum()`: counts rows where the stored Total disagrees with Quantity x Price. (Zero here, but the question asks you to *check*, so printing the count shows the marker you did.)

- `df["Total"] = df["Quantity"] * df["Price"]` : overwrite every Total with the correct value. Analogy: the till recalculating the bill.
- `df.drop_duplicates()` : removes rows that are exact copies of an earlier row (keeps the first). Analogy: throwing away photocopies of a receipt. We store `before` so we can print how many were removed (6).

> **Mark-saving tip:** put a short comment above every step using the *wording of the question* ("# Load the data", "# Convert Date to datetime"). Markers scan for these.

4. Task 1b - Two charts (4 marks)

```
qty_per_product = df.groupby("Product")["Quantity"].sum()

plt.figure(figsize=(8, 5))
plt.bar(qty_per_product.index, qty_per_product.values, color="steelblue")
plt.title("Product Sales Distribution")
plt.xlabel("Product")
plt.ylabel("Total Quantity Sold")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

- `df.groupby("Product")["Quantity"].sum()`: **sort the receipts into one pile per product, then add up Quantity in each pile.** Result: a Series whose *index* is the product names and whose *values* are the totals.
- `plt.figure(figsize=(8, 5))`: put a **fresh sheet of paper** on the desk (width 8 in, height 5 in). Always start each chart with it, otherwise the second chart is drawn on top of the first.
- `plt.bar(x_labels, heights)`: bar chart. `.index` = the labels, `.values` = the bar heights.
- `plt.title / xlabel / ylabel`: the **marks are for these**, never skip them.
- `plt.xticks(rotation=45)`: tilt the labels so "Orange Juice" does not collide with its neighbour.
- `plt.tight_layout()`: shrinks the margins so nothing is cut off. `plt.show()`: **hang the finished chart on the wall** (opens the window; close it to let the script continue).

```
df_2023 = df[df["Date"].dt.year == 2023]                # keep 2023 rows
monthly_sales = df_2023.groupby(df_2023["Date"].dt.month)["Total"].sum()

plt.figure(figsize=(8, 5))
plt.plot(monthly_sales.index, monthly_sales.values, marker="o")
plt.title("Total Sales per Month (2023)")
plt.xlabel("Month")
plt.ylabel("Total Sales")
plt.xticks(monthly_sales.index)
plt.grid(True)
plt.tight_layout()
plt.show()
```

- `df["Date"].dt.year`: the `.dt` "accessor" opens up the date column so you can ask for its `.year`, `.month`, `.day`. It only works **because** Task 1a made Date a real date (that is why the paper orders the tasks this way).
- `df[df["Date"].dt.year == 2023]`: **boolean filter**. Inner part gives True/False per row; the outer `df[...]` keeps only the True rows. Note `==` (compare), not `=` (assign).
- `.groupby(df_2023["Date"].dt.month)["Total"].sum()`: pile by month number (1, 2, 3, 4), add up Total per pile. The data stops in April 2023, so you get 4 points: that is correct, do not invent months.
- `plt.plot(x, y, marker="o")`: line chart; `marker="o"` puts a dot on each month so a 4-point line still looks like data. `plt.xticks(monthly_sales.index)` forces whole-number months (otherwise you may see 1.5, 2.5).

> **TRAP (marks!):** the paper says "over the months of **2023**". If you group by month **without** filtering the year, January 2022, 2023 and 2024 are all mixed into one pile. Always filter first.

5. Task 2a - Create the database and insert, no duplicates (9 marks)

```
conn = sqlite3.connect("SalesDB.db")
cursor = conn.cursor()

# --- Create the table Sales (3 marks) ---------------------------------
# UNIQUE(...) is what really stops duplicates - INSERT OR IGNORE alone
# does nothing unless the table has a UNIQUE / PRIMARY KEY rule to break.
cursor.execute("""
    CREATE TABLE IF NOT EXISTS Sales (
        Date     TEXT,
        Product  TEXT,
        Quantity INTEGER,
        Price    REAL,
        Total    REAL,
        UNIQUE (Date, Product, Quantity, Price, Total)
    )
""")
```

Analogy - SQLite is a filing cabinet that is one file. `sqlite3.connect("SalesDB.db")` opens the cabinet (creates the file if it does not exist yet). `cursor` is **the clerk**: you hand them a written instruction with `cursor.execute(...)` and they carry it out. Until you call `conn.commit()` the clerk has only put the papers on their desk; **commit = actually filing them.**

- `CREATE TABLE IF NOT EXISTS Sales (...)`: make a drawer called Sales *unless it already exists* (so re-running your script does not crash).
- Column types: `TEXT` (words and dates), `INTEGER` (whole numbers), `REAL` (decimals).
- `UNIQUE (Date, Product, Quantity, Price, Total)`: a rule stamped on the drawer: **"no two cards with exactly the same five values."** This is the line that really stops duplicates (see the box below).

```
df_db = df.copy()
df_db["Date"] = df_db["Date"].dt.strftime("%Y-%m-%d")    # SQLite wants text

for _, row in df_db.iterrows():
    cursor.execute(
        "INSERT OR IGNORE INTO Sales (Date, Product, Quantity, Price, Total) "
        "VALUES (?, ?, ?, ?, ?)",
        (row["Date"], row["Product"], int(row["Quantity"]),
         float(row["Price"]), float(row["Total"])))
conn.commit()

cursor.execute("SELECT COUNT(*) FROM Sales")
print("\nRows now in Sales table:", cursor.fetchone()[0])
```

- `df.copy()`: make a photocopy so the next line (turning dates into text) does not damage the original DataFrame you still use for plots.
- `.dt.strftime("%Y-%m-%d")`: turn each date into text `2023-01-05`. SQLite has **no real date type**; it stores dates as text, and `YYYY-MM-DD` sorts correctly and works with `strftime` in queries.
- `for _, row in df_db.iterrows():`: visit the DataFrame one row at a time. `iterrows()` gives two things per row, (row number, the row). `_` means "I don't need the row number".
- `VALUES (?, ?, ?, ?, ?)` + the tuple: **fill-in-the-blank form.** Each `?` is a slot filled by the matching item in the tuple. Never glue values into the SQL with f-strings: a value like `O'Brien` breaks it, and a malicious value could rewrite your query (SQL injection).
- `int(...)`, `float(...)`: converts pandas/numpy numbers into plain Python numbers, which SQLite always accepts. (Safe habit.)
- `INSERT OR IGNORE`: "if this card breaks a UNIQUE rule, quietly skip it instead of raising an error."
- `conn.commit()` once, **after** the loop: faster, and everything is saved together.
- `SELECT COUNT(*)` + `fetchone()[0]`: proves how many rows are stored. `fetchone()` returns a tuple like `(145,)`, so `[0]` takes the number out.

> **WHY the official solution double-inserts (tested!).** The official 2024 solution uses `INSERT OR IGNORE` but its `CREATE TABLE` has **no UNIQUE rule**, so there is nothing to ignore: run it twice and the table grows from 151 to **302 rows** and "total sales" doubles to 17,800. Mine has the UNIQUE line plus `drop_duplicates()`, and gives 145 rows on every run. In the exam, doing it either way will probably earn the marks; doing it properly protects you.

6. Task 2b - Querying (6 marks)

```
# --- Total sales for 2023 (3 marks) ----------------------------------
cursor.execute(
    "SELECT SUM(Total) FROM Sales WHERE strftime('%Y', Date) = '2023'")
total_2023 = cursor.fetchone()[0]
print(f"\nTotal sales in 2023: {total_2023}")

# --- Quantity sold per product in 2023, biggest first (3 marks) -------
cursor.execute("""
    SELECT Product, SUM(Quantity) AS TotalQty
    FROM Sales
    WHERE strftime('%Y', Date) = '2023'
    GROUP BY Product
    ORDER BY TotalQty DESC
""")
print("\nProduct sales summary for 2023:")
for product, qty in cursor.fetchall():
    print(f"  {product:<15} {qty}")

conn.close()
```

Read an SQL query like an English sentence, clause by clause:

SQL	In plain English
`SELECT SUM(Total)`	"give me the grand total of the Total column"
`FROM Sales`	"from the Sales drawer"
`WHERE strftime('%Y', Date) = '2023'`	"only the cards whose year part is 2023". `strftime('%Y', ...)` reads just the year off the date text; note the **quotes around '2023'**, because strftime returns text
`SELECT Product, SUM(Quantity) AS TotalQty`	"show each product and its total quantity, and call that column TotalQty"
`GROUP BY Product`	"make one pile per product" (same idea as pandas `groupby`)
`ORDER BY TotalQty DESC`	"biggest first" (`ASC` is smallest first)

- `cursor.fetchone()` = the **next single row** (a tuple). `fetchall()` = **all rows** as a list of tuples.
- `for product, qty in cursor.fetchall():` unpacks each 2-item tuple into two names.
- `f"{product:<15}"`: left-align in 15 characters so the output lines up in columns.
- `conn.close()`: lock the cabinet. Do this *after* your last query, never before.

Check yourself: total 2023 sales = **8950.0**. Quantity per product 2023: Sweets 180, Orange Juice 152, Biscuits 140, Chocolate 95, Water 60, Unknown 13.

7. Task 3 - The neural network (12 marks)

Analogy - a neural network is a panel of judges. `nn.Linear(10, 5)` = 5 judges, and each one reads all 10 facts about a sample and gives one score. The next layer, `Linear(5, 3)`, has 3 judges who read the first panel's 5 scores. The last, `Linear(3, 1)`, is a single head judge giving the final answer. **ReLU** = "a judge only speaks up if their score is positive; negatives become 0". **Sigmoid** = "squash the head judge's score to a number between 0 and 1".

```
class SimpleNN(nn.Module):
    def __init__(self):
        super().__init__()                  # set up the nn.Module machinery
        self.fc1 = nn.Linear(10, 5)         # 10 input features -> 5 neurons
        self.fc2 = nn.Linear(5, 3)          # 5 -> 3 neurons
        self.fc3 = nn.Linear(3, 1)          # 3 -> 1 output neuron

    def forward(self, x):                   # how data flows through
        x = torch.relu(self.fc1(x))         # hidden layer 1 + ReLU
```

```
        x = torch.relu(self.fc2(x))         # hidden layer 2 + ReLU
        x = torch.sigmoid(self.fc3(x))      # output layer + Sigmoid
        return x
```

- `class SimpleNN(nn.Module):` "my network is a *kind of* `nn.Module`" (inheritance): you get PyTorch's machinery (tracking weights, `.parameters()`) for free.
- `super().__init__()`: **must be the first line.** It runs the parent class's set-up. Forget it and PyTorch raises `AttributeError: cannot assign module before Module.__init__() call`.
- `nn.Linear(10, 5)`: (inputs, outputs). **The OUTPUT of one layer must equal the INPUT of the next** (5 -> 5, 3 -> 3). A mismatch gives `mat1 and mat2 shapes cannot be multiplied`.
- `forward(self, x)`: the **route** a piece of data takes. You never call it directly; calling `model(data)` runs it for you.
- The paper says "Input Layer: 10 features" but the *input layer is not a layer you code*: it is just the 10 in `Linear(10, 5)`.

```
model = SimpleNN()
criterion = nn.MSELoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)

# --- 3b: synthetic data (2 marks) ----------------------------------
data = torch.randn(100, 10)      # 100 samples, 10 features each
target = torch.randn(100, 1)     # 100 target values (shape matches output)
```

- `model = SimpleNN()`: build one network (**3a**).
- `nn.MSELoss()`: Mean Squared Error = average of (prediction - truth)². Analogy: a quiz score where being very wrong costs far more than being slightly wrong.
- `optim.SGD(model.parameters(), lr=0.01)`: the **study strategy**. It is handed every adjustable weight (`model.parameters()`) and a step size `lr=0.01` (learning rate): small steps = careful, slow learning.
- `torch.randn(100, 10)`: 100 rows x 10 columns of random numbers (bell-curve around 0) = 100 fake samples with 10 features (**3b**). `randn(100, 1)` = 100 fake answers. **Shape (100, 1) matches the model's output shape**; with shape `(100,)` PyTorch would broadcast the two and give a wrong, silently different loss.

```
losses = []
for epoch in range(20):
    optimizer.zero_grad()               # 1. clear old gradients
    output = model(data)                # 2. forward pass
    loss = criterion(output, target)    # 3. how wrong are we?
    loss.backward()                     # 4. backpropagation
    optimizer.step()                    # 5. update the weights
    losses.append(loss.item())          # remember the loss as a number
    print(f"Epoch {epoch + 1:2d}/20  loss = {loss.item():.4f}")
```

This five-step loop is the **same in every PyTorch program ever written. Learn it as a chant: zero, forward, loss, backward, step.**

Line	Meaning	Analogy
`optimizer.zero_grad()`	wipe the old gradients	wipe the whiteboard before marking
`output = model(data)`	forward pass: predictions for all 100 samples	students sit the quiz
`loss = criterion(output, target)`	one number: how wrong overall	the quiz score
`loss.backward()`	backpropagation: work out how much each weight contributed to the error	the teacher traces each mistake back to which judge caused it
`optimizer.step()`	nudge every weight a little in the improving direction	the judges adjust their habits
`losses.append(loss.item())`	`.item()` turns a 1-number tensor into a plain Python float	write the score in the register

- One pass through the loop with all 100 samples = **one epoch** (one full revision session). `range(20)` = 20 epochs (**3c**).

- If you forget `zero_grad()` the gradients **pile up** between epochs and training goes wrong with no error message.

```
# --- 3d: plot the loss (3 marks) ------------------------------------
plt.figure(figsize=(8, 5))
plt.plot(range(1, 21), losses, marker="o")
plt.title("Training Loss Over Epochs")
plt.xlabel("Epoch")
plt.ylabel("Loss (MSE)")
plt.grid(True)
plt.tight_layout()
plt.show()
```

- `plt.plot(range(1, 21), losses)`: x = epoch numbers 1 to 20, y = the 20 recorded losses (**3d**). Label the axes and give a title.

> **"Your plot should display the loss decreasing" - the honest truth (tested).** With the paper's exact settings (random targets, plain SGD, `lr=0.01`, 20 epochs) the loss **does go down every epoch but only by about 1-2%** (e.g. 0.7471 -> 0.7403). Matplotlib auto-scales the y-axis so you still see a downward line, but the numbers barely move. That is expected, not a bug: random targets cannot be learned, and the learning rate is small. If an examiner asks, say exactly that, and offer the fix: a *learnable* target (see drill 11 in `2024_twist_drills.py`: loss 0.707 -> 0.160) or a bigger learning rate / `optim.Adam`.

8. Where marks are won and lost (self-check list)

- [ ] Every sub-task has a **comment** quoting the question, and prints something visible.
- [ ] `Date` really is datetime (`print(df.dtypes)` shows `datetime64`).
- [ ] The line chart uses **2023 only**; charts have title + axis labels.
- [ ] DB is called **SalesDB**, table **Sales**, five columns exactly as listed.
- [ ] Re-running the script does not double the rows.
- [ ] Query results are **printed**, with a label ("Total sales in 2023: ...").
- [ ] `super().__init__()` present; layer sizes chain 10 -> 5 -> 3 -> 1; ReLU, ReLU, Sigmoid in `forward`.
- [ ] Training loop has all five steps; loss recorded with `.item()`; 20 epochs.
- [ ] File named `your_name_python_task.py`; runs top to bottom with no error. **Run it once fully before submitting.**

9. Anticipated questions (written or viva-style)

1. **Why does `Date` need converting?** Because `read_csv` loads it as text; text cannot be filtered by year or grouped by month. `to_datetime` makes it a real date, enabling `.dt.year`.
2. **What does `dayfirst=True` do?** Reads `05/01/2023` as 5 January, not May 1. Needed because the file is UK `dd/mm/yyyy`.
3. **Difference between `fillna(0)` and filling Price from Total/Quantity?** The first loses information; the second recovers the true value.
4. **What does `inplace=True` do and why avoid it?** Modifies the table directly, returns None. In newer pandas, `df["col"].fillna(x, inplace=True)` stops working on a column (it edits a temporary copy). Assigning back (`df["col"] = df["col"].fillna(x)`) is reliable. (`df.fillna({...}, inplace=True)` on the whole frame is fine.)
5. **What does `commit()` do vs `close()`?** `commit` makes changes permanent; `close` closes the connection. Closing without committing loses the inserts.
6. **Why `?` placeholders?** Safety (SQL injection) and correct handling of quotes.
7. **How do you stop duplicates in a database?** A `UNIQUE` / `PRIMARY KEY` constraint plus `INSERT OR IGNORE` (or check with `SELECT` first), and/or `drop_duplicates()` in pandas first.
8. **Why is the target shape (100, 1)?** To match the output shape of `Linear(3, 1)`.
9. **Why Sigmoid on the output?** Squashes to (0, 1); good for probabilities/binary outputs.
10. **What does `zero_grad()` / `backward()` / `step()` do?** See the table in section 7.
11. **Why `loss.item()`?** Converts the tensor to a Python number so it can be stored and plotted (and doesn't keep the whole computation graph alive).
12. **GROUP BY in pandas vs SQL?** Same idea. `df.groupby("Product")["Quantity"].sum()` is `SELECT Product, SUM(Quantity) FROM Sales GROUP BY Product`.

10. Twist drills: practise the variations

Open `Solutions_py/2024_twist_drills.py` (all tested). The likeliest twists, with one-liners:

If the examiner asks for...	Use
revenue per product, best first	`df.groupby("Product")["Total"].sum().sort_values(ascending=False)`
the single best product	`.idxmax()` on that Series, or SQL `... ORDER BY Revenue DESC LIMIT 1`
average price per product	`df.groupby("Product")["Price"].mean()`
sales per month in SQL	`SELECT strftime('%Y-%m', Date) AS Month, SUM(Total) FROM Sales GROUP BY Month ORDER BY Month`
a year held in a variable	`cursor.execute("... WHERE strftime('%Y', Date) = ?", (str(year),))`
rows meeting two conditions	`df[(df["Year"] == 2023) & (df["Total"] > 100)]` (use `&`, brackets round each test)

If the examiner asks for...	Use
query result as a DataFrame	`pd.read_sql_query("SELECT ...", conn)`
whole DataFrame into SQLite	`df.to_sql("Sales", conn, if_exists="replace", index=False)`
pie chart	`plt.pie(series.values, labels=series.index, autopct="%1.1f%%")`
save cleaned data	`df.to_csv("cleaned_sales.csv", index=False)`
different network, e.g. 8 -> 4 -> 1, BCELoss, Adam	`nn.Sequential(nn.Linear(8,4), nn.ReLU(), nn.Linear(4,1), nn.Sigmoid())`, `nn.BCELoss()`, `optim.Adam(...)`

11. Quick-reference box (copy onto your notes sheet)

```
READ     df = pd.read_csv("f.csv");  df.isna().sum();  df.duplicated().sum()
FILL     df["c"] = df["c"].fillna(v)      # ALWAYS assign back
DATE     df["Date"] = pd.to_datetime(df["Date"], dayfirst=True)
FILTER   df[df["Date"].dt.year == 2023]
GROUP    df.groupby("P")["Q"].sum()
CHART    plt.figure(); plt.bar/plot(x, y); title; xlabel; ylabel; show()
DB       conn = sqlite3.connect("X.db"); cur = conn.cursor()
         cur.execute("CREATE TABLE IF NOT EXISTS T (a TEXT, UNIQUE(a))")
         cur.execute("INSERT OR IGNORE INTO T VALUES (?)", (v,)); conn.commit()
QUERY    ... WHERE strftime('%Y', Date) = '2023' GROUP BY P ORDER BY s DESC
NN       super().__init__(); nn.Linear(in, out); forward() with relu/sigmoid
LOOP     zero_grad -> model(x) -> loss -> backward -> step ; losses.append(loss.item())
```